

第十章：数据库恢复技术 —— 故障之后怎么把数据救回来

根本矛盾

数据库对用户承诺：你的数据一定是"对的、完整的、提交了就永久的"。

但现实很残酷：硬件会坏、断电、磁头会撞、程序会崩、操作员会误操作。

第十章要解决的核心问题：故障发生后，怎么把数据库从"被破坏"状态拉回到"正确"状态？

要回答这个问题，需要先想清楚 3 件事：

1. 怎么定义"对" —— 事务 (Transaction)，以及 ACID 4 特性

2. 会出什么问题 —— 4 种故障

3. 怎么救 —— 3 种冗余数据 + 不同故障对应的恢复策略 + 检查点优化

第十一章讲"并发控制"实现 I (隔离性)；第十章讲"恢复技术"实现 A (原子性) 和 D (持续性)；C (一致性) 是 A 的必然结果。本章是 ACID 4 特性中 A 和 D 的具体落地。

ACID 4 特性在数据库课程中的实现分工：

- A 原子性 ——→ 第十章：恢复技术 (UNDO/REDO 机制)
- C 一致性 ——→ A 的必然结果 (不允许"部分提交" → 不会"部分不一致")
- I 隔离性 ——→ 第十一章：并发控制 (封锁/2PL/可串行化)
- D 持续性 ——→ 第十章：恢复技术 (日志+后备副本保障"提交即永久")

一、事务 —— 恢复的"最小单位"

1.1 什么是事务

事务 = 用户定义的一组数据库操作序列，这些操作要么全做、要么全不做，是不可分割的工作单位。

一个事务可以是一条 SQL、一组 SQL，或者整个程序段。但要注意：

事务 ≠ 程序。一个程序 (main 函数) 里可以包含多个事务，每个事务以 BEGIN TRANSACTION 开始，以 COMMIT 或 ROLLBACK 结束。

典型误解：很多人以为"程序就是事务"。其实程序是代码的整体结构，事务是数据库操作的逻辑单位——两个东西层次不同。



```

C 程序 (main 函数)
|
| BEGIN TRANSACTION
|   UPDATE ...      | 第 1 个事务
|   INSERT ...      |
|   COMMIT
|
| BEGIN TRANSACTION
|   DELETE ...      | 第 2 个事务
|   ROLLBACK
|

```

第 8 章的嵌入式 SQL 程序里，整个 main 函数通常就是 1 个事务——因为嵌入式 SQL 没有"事务结束"标志，整个程序跑完才算结束。这是个特例。

1.2 事务的两重身份

源材料强调事务同时是两类机制的基本单位：

身份	作用	范围
恢复的基本单位	故障恢复时，事务要么整个撤销、要么整个重做——不能"半个"	本章
并发控制的基本单位	多事务并发时，按"事务"粒度相互隔离	第十一章

换句话说：事务是数据库"做事"的最小单元——任何机制（恢复、并发）都按事务切分，不会切到事务"中间"。

二、ACID 4 特性 —— 事务的"四大承诺"

数据库承诺的"对、完整、互不干扰、永久"，翻译成技术语言就是 ACID 4 特性。这是期末必背的高频考点。

2.1 ACID 全表

特性	中文	含义	大白话	银行转账类比
A	原子性 (Atomicity)	事务"要么全做、要么全不做"	一笔交易只有 2 种结局——完全成功或完全失败	转账 = "扣 A 100 + 加 B 100"，必须同时成功或同时失败
C	一致性 (Consistency)	事务执行前后，数据库从一个一致状态变到另一个一致状态	"账本对得上"——钱守恒、规则满足	A 扣 100 + B 加 100，钱总和不变
I	隔离性 (Isolation)	多个事务并发执行时互不干扰	多笔交易互相看不到对方的"半成品"	多人同时转账时，不会基于"错的余额"决策
D	持续性 (Durability)	事务一旦提交，对数据库的修改永久生效	提交后永不丢失	转账成功后断电也不丢



2.2 4 个特性之间的关系 —— 为什么"一致性"要单独说

很多同学困惑：原子性已经保证"全做或全不做"了，为什么还要单独说"一致性"？这俩不是一回事吗？

关键区分：

- 原子性关心的是事务本身的"全做或全不做"
- 一致性关心的是整个数据库的"从一个一致状态到另一个一致状态"

两者的逻辑链：

如果允许"部分提交"（违反原子性）→ 已做的那部分已经修改数据库 → 数据库就可能停在"半对半错"的中间状态 → 违反一致性。

>

反过来，只要强制原子性（不允许部分提交）→ 不会出现"半对半错" → 数据库总是在一致状态之间跳转。

>

所以：一致性 = 原子性的必然结果。源材料第 285 行原文："如果事务被迫中断……就会造成数据库处于不一致的状态"——指的就是"原子性破了，一致性跟着破"。

为何还要单独说"一致性"？

因为有些业务规则的一致性不靠"全做或全不做"就能保证——比如"所有学生的年龄必须 ≥ 0 "。即使每个事务都"全做或全不做"了，如果某个事务做完了但写入了非法年龄，数据库还是不一致。这种一致性要靠完整性约束（第 5 章）来保证，不是本章重点。

2.3 ACID 4 字母速记口诀

A 原子"全做/不做"，C 一致"钱守恒"，I 隔离"互不扰"，D 持续"永不丢"

三、事务的 SQL 定义 —— 3 个语句

每个事务用 3 个 SQL 语句标记边界：

语句	作用	银行转账类比	事务结束了吗？
BEGIN TRANSACTION	事务开始	"开始转账"	—
COMMIT	提交——事务的所有操作永久生效	"转账成功，到账了"	结束
ROLLBACK	回滚——撤销已做的所有操作，回到事务开始的状态	"转账失败，恢复原状"	结束

关键认知：

- COMMIT 之后事务就结束了——后续要做事必须开新事务
- ROLLBACK 之后事务也结束了——同理
- 这两个语句都意味着"这个事务完事了"——只是"成功完事"和"失败完事"的区别

四、4 种故障 —— "灾难片清单"

数据库可能出 4 种故障，严重程度递增：

故障	别名	性质	典型例子	损坏范围
----	----	----	------	------



① 事务内部故障	—	事务自己出错	运算溢出、违反完整性约束、参数非法、逻辑错误	1 个事务
② 系统故障	软故障 (Soft Crash)	OS / DBMS 软件崩溃，内存数据丢失	突然断电、OS 蓝屏、DBMS 代码 Bug、CPU 故障	所有未完成事务
③ 介质故障	硬故障 (Hard Crash)	外存（磁盘）物理损坏	磁盘损坏、磁头碰撞、 强磁场、火灾	整个数据库
④ 计算机病毒	—	恶意程序破坏数据	病毒删表、勒索软件加密、SQL 注入破坏	不定（视破坏范围）

4.1 软故障 vs 硬故障 —— 本章最关键的概念区分

这两个故障的区分直接决定恢复策略：

维度	软故障（系统故障）	硬故障（介质故障）
坏了什么	内存 + CPU 状态	磁盘数据
磁盘上的数据库文件	没坏（只是没写完）	可能坏了
丢失了什么	内存里的数据 + 正在运行事务的"未提交修改"	之前所有已落盘的数据
恢复靠什么	日志文件（不需要后备副本）	后备副本 + 日志文件（必须重装）
恢复速度	快	慢（要从备份介质加载）

恍然大悟点：软故障之所以"软"，是因为磁盘上的数据还在——只是没及时写盘；硬故障之所以"硬"，是因为数据物理上被破坏了——必须从备份还原。

>

这个区分在第 5 节（恢复策略）会直接体现：软故障 → 只需要日志；硬故障 → 必须后备副本 + 日志。

4.2 事务内部故障的特点

事务内部故障不依赖外部事件——是事务自己"做错了"。

- 预期故障：程序预见到，提前写 ROLLBACK（比如余额不足就回滚）
- 非预期故障：程序没预料到，事务跑到一半崩了（比如除零、数组越界）

两种都会让事务"未完成"——所以恢复策略是 UNDO（撤销事务已做的修改），第 5 节细讲。

五、恢复机制 —— 怎么"未雨绸缪"

5.1 恢复的"基本原理"

源材料第 302-304 行原文：

恢复机制涉及两个关键问题：

1. 如何建立冗余数据
2. 如何利用这些冗余数据实施数据库的恢复

>

恢复的基本原理：利用存储在后备副本、日志文件和数据库镜像中的冗余数据来重建数据库。

3 种冗余数据源：

冗余数据	是什么	谁负责建立	用于恢复什么故障
后备副本	数据库的"定期备份文件" (DBA 手工 / 自动脚本)	DBA 通过数据转储	介质故障 (数据库坏掉了, 从备份还原)
日志文件	事务对数据库所有更新操作的"流水账"	DBMS 自动登记	事务 / 系统故障 (必须日志才能恢复)
数据库镜像	数据库的"实时复制副本" (另一块磁盘)	DBMS 自动维护	介质故障预防 (主盘坏了立即切换)

"3 种冗余数据 + 不同故障的恢复策略" 是本章后半段的核心。

- 第 5.2 节讲数据转储 (后备副本怎么建)
- 第 5.3 节讲日志文件 (日志怎么记、有什么用)
- 第 5.4 节讲不同故障的恢复策略
- 第 5.5 节讲数据库镜像
- 第 5.6 节讲检查点恢复 (优化)

5.2 数据转储 —— "拍快照"

数据转储 = DBA 定期把整个数据库 (或部分) 复制到磁带 / 另一块磁盘上保存。复制出来的数据叫后备副本 (或后援副本)。

两个维度的分类:

维度	静态转储	动态转储
转储时是否允许事务运行	不允许 (系统静止)	允许 (边备份边用)
优点	数据一致性好	不影响业务 (不用停机)
缺点	必须停服	转储期间数据库还在变, 后备副本可能"过时"
恢复时需不需要日志	不需要 (副本就是某一时刻的"完整一致状态")	必须 (副本 + 日志才能恢复到故障点)

维度	海量转储	增量转储
每次转储什么	整个数据库	只转上次转储后改过的数据
优点	恢复简单 (一套副本搞定)	速度快、空间省
缺点	慢、占空间	恢复时要按顺序合并多次增量

2×2 组合 = 4 种转储方法 (高频考点):

组合	是否停服	是否全量	恢复时需要日志吗
动态海量转储	否	全量	是
动态增量转储	否	增量	是
静态海量转储	是	全量	否 (副本已是一致状态)
静态增量转储	是	增量	否 (同上)

恍然大悟点: "是否需要日志" 取决于 "动态还是静态", 和 "海量/增量" 无关。

- 静态转储期间没有事务运行, 后备副本就是某一刻的完整一致状态——直接用副本即可, 不用日志。
- 动态转储期间事务还在跑, 后备副本可能是"过时的"——必须结合日志才能恢复到故障点。

>



所以"动态转储必须建日志"是铁律，源材料第 309 行明文。

5.3 登记日志文件 —— "记账本"

日志文件 = 记录事务对数据库更新操作的文件，相当于"流水账"。

5.3.1 两种日志格式

格式	记录内容	大白话	占用空间	恢复精度
以记录为单位	事务标识、操作类型、操作对象、更新前旧值、更新后新值	银行流水的每笔交易详情（谁、什么时候、动了什么、改前是多少、改后是多少）	大	高（可精确到字段）
以数据块为单位	事务标识、被更新的数据块	"今天动了哪些账本"（只记哪个账本被翻过）	小	低（只能精确到块）

特殊值约定：

- 插入（INSERT）时"旧值"字段为空
- 删除（DELETE）时"新值"字段为空

5.3.2 日志文件的 3 条作用（高频考点，必背）

源材料第 309-314 行明文：

#	作用	解释
1	事务故障恢复和系统故障恢复必须使用日志文件	没日志救不回来——因为要 UNDO / REDO 都得知道"事务干了啥"
2	动态转储方式中必须建立日志文件	见 5.2 节恍然大悟点——动态转储副本过时，必须日志补齐
3	静态转储方式中也可以建立日志文件	装了副本后，再配合日志做 REDO，能恢复到故障点（更完整）

5.3.3 登记日志的两条原则（必背）

#	原则	为什么
1	登记的次序严格按并发事务执行的时间次序	多个事务并发时，日志必须保持因果序——先发生的事务的日志必须先记，否则恢复时因果错乱
2	必须先写日志文件，后写数据库	核心铁律——保证"日志里记了，但数据库还没改"的中间状态可恢复

为什么"先写日志"反了会出事？

比喻：银行出纳——"先在账本上记一笔（写日志），再把钱给客户（写数据库）"。

>

如果反了："先把钱给客户，再记账"——万一中间断电，钱给出去了但账本没记——钱去哪儿了？

>

"先写日志"保证：即使写数据库前断电，日志里也已经记了"要改什么"——恢复时按日志来改，保证不丢。

为什么"按时间次序"反了会出事？



假如 T1、T2 并发执行，T1 先开始、T2 后开始。如果日志写成 T2 的日志在 T1 前面，恢复时按日志回放——就会先看到 T2 的操作、后看到 T1 的——因果错乱。

六、不同故障的恢复策略 —— "具体怎么救"

4 种故障对应不同恢复策略（源材料核心考点）：

故障	恢复策略	谁来执行	用什么冗余数据
① 事务故障	UNDO（撤销）此事务	系统自动，对用户透明	日志文件
② 系统故障（软）	UNDO 未完成事务 + REDO 已完成事务	系统重启时自动，不需要用户干预	日志文件
③ 介质故障（硬）	重装数据库 + REDO 已完成事务	DBA 手动 + 系统配合	后备副本 + 日志文件
④ 计算机病毒	同介质故障（视破坏程度）	视情况	后备副本 + 日志

6.1 UNDO vs REDO —— 本章最易混的概念

操作	含义	适用场景	关键字
UNDO（撤销）	把事务已经做的修改改回去	事务没干完（未提交就被打断）	"做了 → 改回原样"
REDO（重做）	把事务已经做的修改再做一遍	事务已提交，但修改可能没落盘（断电导致）	"做了 → 没落盘 → 再做一次"

速记口诀：

- 事务没干完 → UNDO（撤销）
- 事务已提交但可能没落盘 → REDO（重做）

>

关键反例："事务已提交且已落盘"既不 UNDO 也不 REDO——数据已经对了，不用动。

6.2 事务故障的恢复流程

事务故障恢复的算法（伪代码）：

1. 反向扫描日志文件（从最后往前找）
2. 找到该事务的所有更新操作
3. 对每个更新操作：
用日志中的"旧值"去恢复数据库
（相当于把"新值"改回"旧值"）
4. 直到找到该事务的 BEGIN TRANSACTION 标记
5. 事务恢复完成



例：事务 T 修改 A=10→20 时崩了。日志里记着 "A: 旧值 10, 新值 20"。恢复时反向扫描日志，用旧值 10 把 A 改回去——这就是 UNDO。

6.3 系统故障的恢复流程

系统故障恢复的算法（伪代码）：

1. 正向扫描日志文件（从头往后找）
2. 找出哪些事务在故障前未完成 → 加入 UNDO 队列
3. 找出哪些事务在故障前已提交但可能未落盘 → 加入 REDO 队列
4. 对 UNDO 队列的每个事务：
反向扫描日志，用"旧值"恢复
5. 对 REDO 队列的每个事务：
正向扫描日志，用"新值"重做

为什么系统故障需要 REDO?

系统故障（断电）的特点是：事务可能提交了，但修改没来得及写盘（内存数据丢了）。

>

举例：T 已 COMMIT，但缓冲区的"已修改数据块"还没刷到磁盘 → 断电 → 重启时磁盘上的数据是"修改前"的旧值，但日志里记着"已提交"和"新值"。

>

必须 REDO（用新值再写一次）才能让数据库追上 T 的提交。

为什么系统故障需要 UNDO?

有些事务在断电时还没提交——但事务的"未提交修改"可能已经写到了磁盘（DBMS 允许事务执行时就刷盘，不必等提交）。

>

这些事务的修改必须撤销（用旧值改回去）——否则数据库里留着"未提交的脏数据"。

6.4 介质故障的恢复流程

介质故障恢复的算法（伪代码）：

1. DBA 装入最近一次的后备副本
(数据库恢复到"最近一次转储时"的状态)
2. 装入后备副本之后到故障点之间的日志文件
3. 重做 (REDO) 日志中已提交的事务
(因为装副本后做的工作要追上)



- 介质故障 vs 系统故障的恢复本质区别：
- 系统故障：磁盘数据还在，只需要日志做 UNDO+REDO
 - 介质故障：磁盘数据可能坏了，必须先装后备副本，再 REDO

七、检查点恢复 —— "跳过无关部分"

7.1 问题：恢复的"代价"

如果日志文件很长（比如跑了一年的数据库），故障恢复时对所有事务都 REDO/UNDO 非常慢。能不能只处理必要的事务？

7.2 解决方案：检查点

检查点 = 在日志文件中打一个标记，标记之前的日志可以"安全丢弃"——因为那些事务的修改已经写入磁盘了。

类比：学期结束前，老师在日志上写"今天 5/20 之前的所有作业都已批改入库"——5/20 之前的日志可以删了（因为批改结果都在成绩册里），5/20 之后的才需要重新检查。

7.3 故障时不同事务的 4 种状态

设检查点时刻为 T_c ，故障发生时刻为 T_f 。事务有 4 种可能状态（用 T_1 、 T_2 、 T_3 、 T_4 表示）：



事务	状态	恢复策略	原因
T_1	在检查点之前开始并完成	不处理	修改早已落盘，不需要 REDO
T_2	在检查点之前开始，检查点之后才完成	REDO（重做）	检查点后到故障前之间可能没全部落盘
T_3	在检查点之前开始，故障时还没完成	UNDO（撤销）	事务没提交
T_4	在检查点之后开始，故障时还没完成	UNDO（撤销）	事务没提交

7.4 速记口诀

- 检查点之前完成 → 不重做（早就落盘了）
- 检查点之后完成 → 重做（REDO）
- 故障时未完成 → 撤销（UNDO）

7.5 引入检查点后的恢复算法

引入检查点的恢复算法（伪代码）：

1. 在日志中找到最后一个检查点记录的位置
2. 从该检查点开始正向扫描日志
3. 建立两个队列：
 - UNDO 队列：在检查点之后开始、故障时未完成的事务
 - REDO 队列：在检查点之后开始、故障时已提交的事务
4. 对 UNDO 队列反向扫描做 UNDO
5. 对 REDO 队列正向扫描做 REDO

关键点：检查点之前的事务完全不管（既不 UNDO 也不 REDO）——因为它们早已落盘，对数据库的修改"安全"。

八、数据库镜像——"实时备份"

8.1 概念

数据库镜像 = DBMS 自动把整个数据库的数据复制到另一块磁盘。一旦主磁盘坏了，备用磁盘上的数据是最新的，立即接管。

源材料第 339 行原文（带 * 号）："为避免磁盘介质出现故障影响数据库，许多 DBMS 提供了数据库镜像功能"。

8.2 数据库镜像 vs 数据转储 vs 日志

机制	性质	触发时机	恢复速度	适用
数据转储（后备副本）	事后补救	故障发生后用副本还原	慢（要装入备份）	介质故障
日志文件	辅助补救	配合后备副本，事后追到故障点	快（无需装副本）	事务 / 系统故障
数据库镜像	事中预防	故障发生前实时复制到备用盘	极快（备用盘直接切换）	介质故障预防

三者的关系：

- 数据转储 + 日志文件 = "事后补救"（出事了再救）
- 数据库镜像 = "事中预防"（出事时备用盘顶上）

>

数据库镜像不能替代转储和日志——它只防"磁盘突然坏了"，不防"误操作"（你把主盘的数据误删了，备用盘也会被同步删掉）。

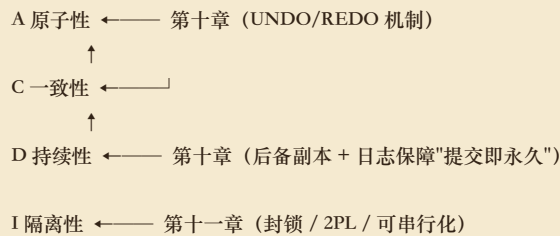


九、跨章呼应

第十章是"恢复技术"，与前后章节的连接：

关联章节	关联点
第 1 章 绪论	DBMS 统一管理的 4 大任务之一 = 恢复技术（本章）
第 4 章 安全性	并发访问时，每个用户仍要符合授权（恢复后数据库的访问权限不丢）
第 5 章 完整性	完整性检查 vs 恢复：5 章管"数据对不对"，10 章管"数据丢了能不能找回来"
第 7 章 数据库设计	维护阶段"转储和恢复" = 本章内容（数据转储 + 恢复策略）
第 8 章 数据库编程	COMMIT / ROLLBACK = 事务的提交 / 回滚（嵌入式 SQL 程序 = 一个或多个事务）
第 11 章 并发控制	撤销（UNDO）= 死锁解除 / 检查点恢复 / 事务回滚——都靠"撤销"操作；ACID 的隔离性 = 11 章目标

跨章视角下的 ACID 实现：



易混淆对比总结

表 1：ACID 4 特性

特性	含义	一句话	谁来保证
A 原子性	全做或全不做	不可分割	本章（恢复技术）
C 一致性	钱守恒、规则满足	状态正确	A 的必然结果 + 完整性约束
I 隔离性	多事务互不干扰	互相看不见半成品	第 11 章（并发控制）
D 持续性	提交后永久不变	永不丢失	本章（恢复技术）

表 2：4 种故障

故障	别名	严重度	恢复策略	用什么冗余数据
事务内部	—	轻	UNDO	日志
系统	软故障	中	UNDO + REDO	日志
介质	硬故障	重	重装 + REDO	后备副本 + 日志
计算机病毒	—	重	同介质	后备副本 + 日志



表 3：软故障 vs 硬故障

维度	软故障（系统）	硬故障（介质）
坏了什么	内存 + CPU 状态	磁盘数据
磁盘数据库文件	没坏	可能坏了
恢复靠什么	日志（不需要后备副本）	后备副本 + 日志
恢复速度	快	慢

表 4：数据转储的 4 种组合

组合	是否停服	是否全量	恢复时需要日志吗
动态海量	否	全量	是
动态增量	否	增量	是
静态海量	是	全量	否
静态增量	是	增量	否

表 5：日志文件的 2 种格式

格式	记录内容	类比
以记录为单位	事务 / 操作类型 / 对象 / 旧值 / 新值	银行流水每笔详情
以数据块为单位	事务 / 被更新的数据块	只记"动了哪些账本"

表 6：UNDO vs REDO

操作	含义	适用
UNDO（撤销）	把已做的修改改回去	事务没完成
REDO（重做）	把已做的修改再做一遍	事务已提交但可能没落盘

表 7：检查点恢复的 4 种事务状态

状态	例子	恢复策略
检查点前完成	T1	不处理
检查点后完成	T2	REDO（重做）
故障时未完成（检查点前开始）	T3	UNDO（撤销）
故障时未完成（检查点后开始）	T4	UNDO（撤销）

表 8：3 种冗余数据机制对比

机制	性质	触发时机	适用故障
后备副本（数据转储）	事后补救	故障发生后用	介质故障
日志文件	事后追到故障点	配合后备副本 / 单独用	事务 / 系统故障
数据库镜像	事中预防	故障发生前实时复制	介质故障预防

逻辑主线



数据库对用户的"承诺" vs 现实的"故障"



[怎么定义"对"] 事务

- ├— 事务 vs 程序（一个程序可有多个事务）
- ├— 事务的两重身份（恢复的基本单位 + 并发控制的基本单位）
- └— SQL 定义：BEGIN TRANSACTION / COMMIT / ROLLBACK



[事务的承诺] ACID 4 特性

- ├— A 原子性（不可分割）
- ├— C 一致性（A 的必然结果）
- ├— I 隔离性（第 11 章）
- └— D 持续性（永不丢失）



[会出什么错] 4 种故障

- ├— 事务内部（自己出错）
- ├— 系统软故障（断电、OS 崩）——磁盘数据还在
- ├— 介质硬故障（磁盘坏）——数据物理损坏
- └— 计算机病毒



[怎么未雨绸缪] 3 种冗余数据

- ├— 机制一：数据转储
 - | ├— 静态 vs 动态（是否停服）
 - | ├— 海量 vs 增量（是否全量）
 - | └— 4 种组合（恢复时是否需要日志）
- |
- ├— 机制二：登记日志文件
 - | ├— 2 种格式（记录 / 数据块）
 - | ├— 3 条作用（事务 / 系统恢复必须 / 动态转储必须建 / 静态转储也能建）
 - | └— 2 条原则（按时间次序 / 先写日志后写数据库）
- |
- └— 机制三：数据库镜像（事中预防 vs 事后补救）



[具体怎么救] 不同故障的恢复策略

- ├— 事务故障 → UNDO（伪代码：反向扫描，用旧值改回）
- ├— 系统故障 → UNDO + REDO（伪代码：双向扫描，分别处理）
- └— 介质故障 → 重装 + REDO（伪代码：装副本后追日志）



[效率优化] 检查点恢复

- ├— 跳过无需 REDO 的事务
- └— 4 种事务状态（T1 不处理 / T2 REDO / T3 T4 UNDO）



跨章呼应：第 1 章 DBMS 4 大任务 / 第 5 章完整性 / 第 7 章维护阶段 / 第 8 章 COMMIT/ROLLBACK / 第 11 章隔离性 + UNDO

整个第十章就一件事：故障不可避免，但通过"事务 + ACID 4 特性 + 3 种冗余数据 + 不同故障的恢复策略 + 检查点优化"，让数据库重新对齐"承诺"和"现实"——具体实现 ACID 中的 A（原子性）和 D（持续性），并由此保证 C（一致性）。

